SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Topic : :Design and Development of a Behaviour-Responsive 2D Survival Horror Game Using C++

DSA0105- Object Oriented Programming with C++

Guided By:

Jegadeesh. K

Presented By:

Rashmi Naurene(192521357)

Nikitha R (192524021)

Roshini B T(192524064)

Abstract

- The project develops a 2D survival horror game using C++.
- The player explores a dark environment while avoiding a dangerous enemy.
- The enemy changes its behaviour based on player actions and movement.
- The game includes health, survival objectives, obstacles, and limited resources.
- The project demonstrates game development concepts such as artificial intelligence, collision detection, and state-based behaviour.

Problem Statement

- Traditional 2D horror games may use predictable enemy movement.
- Predictable enemy behaviour reduces suspense and replayability.
- Players need a more challenging experience in which enemies react to their decisions.
- Developing adaptive enemy behaviour requires an efficient and understandable game logic system.
- This project addresses the problem by implementing a behaviour-responsive enemy using C++.

Objectives

- To design and develop a playable 2D survival horror game using C++.
- To implement enemy AI that reacts to player actions.
- To create player movement, health management, and collision detection.
- To design a dark environment with obstacles, collectibles, and survival objectives.
- To improve suspense and replayability through dynamic enemy behaviour.

Proposed System

- The proposed system uses C++ to develop an interactive 2D survival horror game.
- The player can move, explore the map, collect resources, and avoid enemies.
- The enemy uses behaviour states to respond to the player's actions.
- The system includes collision detection, health tracking, and win/lose conditions.
- A game loop continuously updates player movement, enemy decisions, and screen rendering.

System Architecture Diagram



Technologies used

- C++ – Core programming language used to develop the game.
- raylib – Graphics and game framework used to render the game and handle player input.
- Finite State Machine (FSM) – Used to manage enemy behaviours such as patrolling, investigating, chasing, and searching.
- A Pathfinding* – Used for enemy navigation towards reachable destinations.
- Rule-Based Game Mechanics – Used to implement stamina, inventory, hiding, objectives, and noise-based interactions.

Module 1 — Player Control and Game Environment

- Implements player movement using keyboard controls (WASD or arrow keys).
- Designs a 2D horror environment with rooms, corridors, and obstacles.
- Uses collision detection to prevent movement through walls.
- Creates a dark atmosphere using lighting and visual effects.
- Supports exploration and interaction with objects such as keys and doors.

Flow diagram:

Player Input → Movement Processing → Collision Detection → Environment Update

Module 2 — Behaviour-Responsive Enemy AI

- Develops enemy intelligence using a Finite State Machine (FSM).
- Implements patrol behaviour when the player is not detected.
- Makes the enemy chase the player when detected.
- Enables searching when the enemy loses sight of the player.
- Changes enemy behaviour according to player movement and actions.

Flow diagram:

Patrol → Detect Player → Chase Player → Search Area → Return to Patrol

Module 3 — Survival Mechanics and Game Management

- Maintains player health and damage levels.
- Tracks collectibles, keys, and survival objectives.
- Implements win and game-over conditions.
- Displays health, objectives, and game status on the screen.
- Provides restart and exit options after the game ends.

Flow diagram:

Start Game → Track Health and Objectives → Check Conditions → Victory / Game Over

Sample Output

Advantages

- Adaptive gameplay: Enemies react to player detection and movement.
- Improved suspense: Unpredictable encounters make the game more engaging.
- Efficient programming: C++ supports fast game logic and real-time updates.
- Modular design: Separate modules simplify testing, debugging, and maintenance.
- Replayability: Different player decisions can lead to different encounters.

Conclusion

- The project proposes a 2D survival horror game developed using C++.
- Behaviour-responsive enemy AI is the main feature of the system.
- Player controls, collision detection, survival mechanics, and game state management work together to create the gameplay.
- The use of a Finite State Machine provides a structured approach to enemy decision-making.
- The project can be extended with advanced AI, new levels, sound effects, and improved visual effects.

Thank you